Scalar-Separator Elimination for Normal-Surface Search

Exact conic messages, source-bound extreme rays, and a quasi-polynomial width criterion

Research continuation prepared for Vladimir Reshetnikov
ProveIt / Topology / UnknotRecognition
Mathematical development and implementation assisted by ChatGPT

12 October 2026

Abstract

The latest ProveIt work separates two tasks that must not be confused: processing a supplied, binary-encoded normal surface and discovering a useful surface. We address discovery. For homogeneous nonnegative matching equations with local incompatibility constraints, we prove that a subtree attached through one signed scalar has an exact positive-objective continuation with only two rational slopes, two availability bits, and a zero-port positive flag. This replaces a product of local compatibility searches by their sum.

A polynomial exact compiler recognizes supplied column partitions whose cut spaces have dimension at most one. A further theorem shows that the resulting flux-sign subdivisions introduce no new extreme rays. Consequently, the implemented ray-gluing procedure returns primitive connected normal witnesses when bound to genuine standard matching equations. We give two explicit bit bounds: one for the delivered rational ray enumerator, and a sharper local intrinsic-dimension bound using a polynomial-time linear-programming primitive. Both yield conditional quasi-polynomial recognition criteria with all source, decomposition, and geometric costs stated.

The package includes 27 passing unit tests, 768 independent global-oracle queries repeated on source-recompiled models, 1,536 certificate replays, 455 source-extremality checks, and actual one- and two-tetrahedron matching fixtures. On the latter, a verified nongeometric column split reduces candidate bases from 3,432 to 1,856 and gives a measured 1.94-fold gain for eight complete anchored Euler queries in the delivered kernel. A two-coordinate interface example proves that a literal finite union of convex cones may already require exponentially many pieces. No unrestricted quasi-polynomial unknot recognizer, native production integration, or universal small-separator theorem is claimed.

Reading guide. Sections 3–5 contain the main proofs. Section 6 distinguishes implemented and theoretical bounds. Section 8 states the exact recognition interface. Section 9 records both improvements and negative findings. The executable package is standard-library Python.

Contents

1 Research position and scope **3**
1.1 The gap addressed here . . . 3
1.2 What is established, and what is not . . . 3
1.3 Provenance and novelty boundaries . . . 3

2 The exact algebraic problem **4**

3 Two slopes suffice at a scalar interface **5**
3.1 Local recurrence . . . 6
3.2 Witness recipes and negative evidence . . . 6

4 Compiling rank-one column cuts from the source **7**
4.1 An exact and reusable source contract . . . 8

5 Source-bound gluing preserves extreme rays **8**

6 Bit complexity and the improvement over global branching **10**
6.1 The delivered enumeration bound . . . 10
6.2 A sharper local intrinsic-dimension theorem . . . 11
6.3 A strict separation from a global search budget . . . 12

7 A sharp warning for two-coordinate interfaces **13**

8 Connection to complete unknot recognition **14**
8.1 The standard-coordinate interface . . . 14
8.2 An explicit conditional running-time criterion . . . 14
8.3 Relation to Lackenby's hierarchy program . . . 15

9 Implementation, verification, and experiments **15**
9.1 Delivered components . . . 15
9.2 Verification design . . . 16
9.3 Independent global-oracle audit . . . 16
9.4 Actual normal-coordinate fixtures . . . 16
9.5 A genuine source split, and a rejected geometric split . . . 17
9.6 Algebraic scaling and small-input overhead . . . 18
9.7 Limits of this validation . . . 18

10 Integration plan and research priorities **19**
10.1 An additive integration path . . . 19
10.2 Further mathematical questions . . . 19

11 Conclusion **21**

A Reproduction and file map **21**

1 Research position and scope

1.1 The gap addressed here

Lackenby's 2021 announcement motivates the quasi-polynomial target. His July 2026 preprint develops the hierarchy framework, while Section 9 explicitly leaves some algorithmic steps insufficiently specified for a running-time estimate [2, 1]. An implementation must therefore distinguish structural mathematics, a complete decision procedure, and a proved end-to-end cost bound. This article contributes to that distinction rather than treating the announcement as a ready-made cost specification.

The inspected ProveIt synthesis already describes a complete raw normal cut with binary affine cell families, signed attachments, and reconstruction charts. Its encoded-size theorem concerns the number of source tetrahedra and coordinate bits, not the number of represented normal discs [8]. We do not repeat that construction. The present question is instead: can matching-cone search be decomposed before its compatibility choices multiply?

Report 87 has already proved a global intrinsic-dimension bound for

$$Ax = 0, \qquad x \geq 0, \qquad c^T x > 0, \qquad x_i x_j = 0 \quad (\{i, j\} \in \mathcal{P}).$$

Its branch tree has at most $2^{d+1} - 1$ nodes, where $d = \dim \operatorname{span}\{x \geq 0 : Ax = 0\}$, assuming polynomial-time rational linear programming [7]. Neither this bound nor complete support reduction is a new claim of this article. Our refinement replaces one large d by many local dimensions separated by exactly checked scalar interfaces.

The main mechanism is elementary but useful. Positive homogeneity means that one nonzero interface coordinate can be normalized to 1 or -1. Once a positive zero-interface direction is excluded, the best objective extension has one finite slope for each attainable sign. The compatibility choices inside a region can then be forgotten without convexifying the entire admissible domain.

1.2 What is established, and what is not

The mathematical results below are unconditional for their specified rational cone and tree inputs. The normal-surface consequences additionally require source-authenticated standard coordinates. The recognition complexity result additionally requires a bounded-cost decomposition producer and the established geometric recognition operations. These are different layers of hypotheses.

The delivered code computes all needed local rays exactly, constructs and checks rank-one source compilations, answers positive-objective queries, and produces independently replayable packets. It does not construct a knot exterior from a diagram, validate all manifold links, perform crushing, or modify the maintained recognizer. The source fixtures exercise actual matching and Euler arithmetic but do not constitute a full knot-corpus experiment.

A supplied decomposition is not a discovered decomposition. Likewise, a small normal-coordinate bit length is not a small scalar-interface width. Both costs remain visible in the statements below.

1.3 Provenance and novelty boundaries

The reference repository checkpoint is

`4840e384b5a2961aabbb9cc78023b9ac5990acb0`.

The report-87 dimension-branching source was re-read at that exact revision. Other reviewed public text included the project README, the report catalogue, the raw-cut manuscript, and the

two-fixture test file. The tree was changing during inspection; the source audit records paths and the relevant returned blob identities instead of implying a complete local checkout.

The four ZIPs listed in `docs/incoming` were inventoried, but their binary contents could not be obtained in this environment. They were not reviewed or executed. This qualification matters when judging overlap with very recent submissions. The standalone delivery does not assign itself a report number or overwrite incoming material.

Rank-one column-space connections belong to the classical setting of matrix and matroid parallel/series connections and 2-sums; this relationship is also explicit in contemporary polyhedral work [6]. We claim neither the invention of such connections nor the invention of dynamic programming, linear programming, or extreme-ray enumeration. The contribution proposed for ProveIt is the precise combination of signed objective messages, source-bound ray preservation, exact compatibility locality, executable source replay, and the resulting local-width cost criterion. Priority of the complete combination has not been established by an exhaustive literature search.

2 The exact algebraic problem

Let $A \in \mathbb{Q}^{m\times n}$, $c \in \mathbb{Q}^n$, and let $\mathcal{P}$ be a list of unordered pairs of distinct coordinate indices. Define

$$C(A) = \{x \in \mathbb{R}^n : Ax = 0,\ x \geq 0\}, \qquad C_{\mathcal{P}}(A) = \{x \in C(A) : x_i x_j = 0\ (\{i,j\} \in \mathcal{P})\}.$$

We decide whether $c^T x > 0$ on $C_{\mathcal{P}}(A)$. All coefficients and integer counts are binary encoded. Zero coordinates specified by the caller are additional coordinate-face restrictions.

The admissible set is a finite union of rational coordinate faces of $C(A)$. It is generally not convex. Each face is a pointed polyhedral cone, so it is generated by finitely many rational extreme rays. Because a coordinate face has no new extreme rays, the admissible set is the union of the cones generated by compatible subsets of the extreme rays of $C(A)$. In particular, a positive objective is witnessed by an admissible rational ray and hence, after scaling, by an integer vector.

This last observation is an existence statement, not an efficient global enumeration algorithm. There may be exponentially many such rays. Our method will enumerate only local rays or, in the stronger theoretical variant, use local compatibility branching instead.

Definition 2.1 (Homogeneous scalar tree model)**.** A rooted finite tree has, at vertex v, a private block $x_v \geq 0$ of original coordinates. Original coordinate blocks are disjoint. Every tree edge e carries one shared signed scalar $\tau_e \in \mathbb{R}$. At vertex v there are rational homogeneous equations

$$B_v x_v + \sum_{e \ni v} h_{v,e}\tau_e = 0$$

and forbidden pairs involving only coordinates of x_v. The objective is $\sum_v c_v^T x_v$. No forbidden pair crosses an edge, and no hidden condition couples two subtrees except through their displayed scalar.

A zero scalar is a valid interface value, and the all-zero assignment is always feasible. These facts are essential. Fixed nonzero boundary data, prescribed integral charges, nonhomogeneous inequalities, or compatibility between private coordinates in different bags are not silently admitted into this definition.

The model is an exact algebraic assembly, not an assertion about cutting a 3-manifold into the corresponding tree pieces. Section 4 gives a source compiler whose algebraic validity is independently checkable.

3 Two slopes suffice at a scalar interface

Consider a subtree T_v, with parent interface τ. Write f_v for the sum of objective contributions of its private coordinates. A compatible assignment means one satisfying all equations, nonnegativity conditions and forbidden pairs inside the subtree.

Definition 3.1 (Continuation message)**.** The zero-port flag Z_v is true when a compatible assignment satisfies $\tau = 0$ and $f_v > 0$. For $s \in \{-1, 1\}$, sign s is available when some compatible assignment has $\tau = s$. When Z_v is false and sign s is available, put

$$\alpha_{v,s} = \max\{f_v : \tau = s, \text{ the subtree assignment is compatible}\}.$$

An unavailable sign is represented by a separate symbol, not by the numerical value zero.

Theorem 3.2 (Exact scalar continuation)**.** *If Z_v is false, each available $\alpha_{v,s}$ is finite, attained, and rational. For every specified scalar t, the best subtree objective is exactly*

$$F_v(t) = \begin{cases} 0, & t = 0, \\ |t|\alpha_{v,\operatorname{sgn} t}, & t \neq 0 \text{ and its sign is available}, \\ -\infty, & t \neq 0 \text{ and its sign is unavailable.} \end{cases}$$

If Z_v is true, the full tree already has a positive assignment, obtained by extending a positive zero-port witness by zero outside the subtree.

Proof. Choose one allowed coordinate face for every local compatibility condition and one sign for every internal scalar. Replacing a scalar of fixed sign by a nonnegative variable gives a rational polyhedral cone. There are finitely many such choices. Intersecting each cone with $\tau = s$ produces a rational polyhedron, possibly empty.

If the objective is unbounded above on a nonempty such polyhedron, linear programming gives a recession direction with parent coordinate zero and positive objective. The direction remains in the same coordinate face and sign chamber, so it is compatible. This would make Z_v true. Thus every nonempty normalized polyhedron has finite optimum. A finite optimum of a rational polyhedron is attained at a rational point. Taking the maximum over the finitely many nonempty choices proves the first assertion.

When $t \neq 0$, multiplication or division by $|t|$ is a bijection between the fiber at $\operatorname{sgn} t$ and the fiber at t, preserving compatibility. The objective scales by $|t|$, giving the displayed formula. At zero, no positive objective is possible by hypothesis, and the zero assignment attains zero. Finally, a zero-parent-port assignment satisfies the outside equations when every outside coordinate and port is set to zero. Homogeneity and the absence of cross-bag conflicts prove the last assertion. □

Corollary 3.3 (Context equivalence for the decision query)**.** *Replacing a subtree by its exact two-slope message preserves whether any homogeneous context attached only at that scalar has positive total objective. The message need not preserve the set of internal surfaces, component counts, full normal coordinates, or arbitrary topological attachment data.*

The theorem is not a convex-hull relaxation. Availability is retained for both signs, the value at zero is treated separately, and the best witness for one sign need not have the same compatibility pattern as the best witness for the other. The reduction is valid because the context sees precisely one scalar and one additive objective contribution.

3.1 Local recurrence

At a vertex v, fix signs $\sigma_e \in \{-1, 1\}$ on all incident edges and write $\tau_e = \sigma_e z_e$, $z_e \geq 0$. The local homogeneous cone in (x_v, z) is

$$K_{v,\sigma} = \left\{ (x_v, z) \geq 0 : B_v x_v + \sum_{e \ni v} \sigma_e h_{v,e} z_e = 0 \right\}.$$

Unavailable child signs force their corresponding z_e to zero. Caller-disabled original coordinates also vanish. These are coordinate faces, not new general inequalities. On this cone the effective local objective is linear:

$$g_{v,\sigma}(x_v, z) = c_v^T x_v + \sum_{e \text{ child of } v} \alpha_{e,\sigma_e} z_e. \tag{1}$$

Terms on forced-zero child coordinates are omitted.

Take every local extreme ray that satisfies the local forbidden pairs and coordinate-zero restrictions. A positive value of $g_{v,\sigma}$ on a ray with zero parent port produces Z_v. Otherwise, for a ray with parent value $z_p > 0$, its candidate parent slope is $g_{v,\sigma}/z_p$. Maximize these candidates separately for the two parent signs. At the root there is no parent port; a positive effective value answers the original query.

Proposition 3.4 (Completeness of the local-ray recurrence)**.** *The recurrence computes the messages of Theorem 3.2 exactly. It remains exact when prepared ray lists are reused after any further original coordinate deletions and any change of the linear objective.*

Proof. In a fixed sign chamber, a compatible vector lies in a coordinate face on which its conflicting coordinates vanish. The extreme rays of this face are extreme rays of the unrestricted local cone and have support contained in that of the vector. They therefore satisfy all its incompatibility and zero conditions.

Express the vector as a nonnegative combination of those rays. If its parent port is zero, every constituent ray has zero parent port, since that coordinate is nonnegative in the chamber. Positive total objective then requires one positive zero-port ray. If its parent port is positive and no positive zero-port ray exists, zero-port terms cannot improve the objective. The ratio of objective to parent port on the remaining sum is a weighted average of the ratios on its positive-port rays. Its maximum is therefore attained by one ray.

By induction, a child slope is attainable by a child assignment with the exact required sign and scale. Independently attaching those child assignments realizes every retained local candidate. No incompatible choice is combined inside a private coordinate block. This proves both inequalities needed for exactness. Further coordinate deletions select additional faces, whose rays are obtained simply by filtering the prepared lists. Objectives change scores, not the underlying rays. □

3.2 Witness recipes and negative evidence

A maximizing message retains one local ray and references the chosen child recipes. To reconstruct a witness, scale a child's stored ray so that its parent scalar equals the requested scalar. Set an entire child subtree to zero when that scalar is zero. A zero-port positive witness found below the root is similarly extended by zero outside its origin subtree.

The implementation uses an explicit stack, not recursion proportional to the height of the input tree. It emits original coordinates and signed edge fluxes. A positive checker needs only exact equation checks, nonnegativity, local pair checks, zero-coordinate checks, and evaluation of the original objective.

For a negative answer, the supplied packet contains complete local ray lists and all finite slopes or unavailable symbols. An independent replay enumerates local positive supports rather than the producer's active zero sets, compares the ray lists, and checks the scalar recurrence. This checker is exponential in local bag size. Its existence is not a polynomial-size negative certificate claim for arbitrary bags. Its support-enumeration cost is bounded by $\mathrm{poly}(N)\sum_v 2^{b_v+2\delta_v}$ with the parameters defined below; it need not be polynomial in the smaller active-zero-set budget alone.

4 Compiling rank-one column cuts from the source

Suppose a rooted tree carries a partition $(I_v)_v$ of the original columns of A. Empty internal bags are allowed. Every forbidden pair must lie wholly in one bag. For a nonroot vertex v, let S_v be the union of the column bags in its rooted subtree. Define

$$\lambda_v = \dim\bigl(\operatorname{im} A_{S_v} \cap \operatorname{im} A_{\overline{S_v}}\bigr) = \operatorname{rank} A_{S_v} + \operatorname{rank} A_{\overline{S_v}} - \operatorname{rank} A. \tag{2}$$

The equality holds because the sum of the two column spaces is $\operatorname{im} A$. The relevant quantity is this column-space intersection dimension, not the number of geometric boundary faces, the graph cut size, or the rank of an arbitrarily selected submatrix of crossing entries.

Assume $\lambda_v \le 1$ on every tree edge. If it equals one, choose a primitive integer generator w_v of the intersection line. If it equals zero, set $w_v = 0$ and require $\tau_v = 0$ at both endpoints. For nonzero w_v, define the signed flux by

$$A_{S_v} x_{S_v} = w_v \tau_v. \tag{3}$$

The scalar can have either sign; replacing it by its absolute value would be an unsound source transformation.

The compiled equation at vertex v is

$$A_{I_v} x_{I_v} + \sum_{u:\mathrm{parent}(u)=v} w_u \tau_u - \begin{cases} w_v\tau_v, & v \neq 0, \\ 0, & v = 0 \end{cases} = 0. \tag{4}$$

The objective and all forbidden pairs are copied to their unique original coordinate bags. No elimination of original coordinates is needed.

Theorem 4.1 (Exact source compilation)**.** *Projection to the original coordinates is a linear bijection from the solution space of the compiled equations to* $\ker A$*. The inverse is given by the unique fluxes in* (3)*, with zero flux on rank-zero cuts. This bijection preserves nonnegativity of original coordinates, every source forbidden pair, original coordinate deletions, and every linear objective on original coordinates. Compilation and independent source verification have polynomial rational bit complexity in the source and supplied-tree encoding.*

Proof. Given $Ax = 0$, the vector $A_{S_v} x_{S_v}$ belongs to $\operatorname{im} A_{S_v}$ and equals $-A_{\overline{S_v}} x_{\overline{S_v}}$, so it lies in the displayed intersection. It therefore determines the unique stated scalar. Partitioning S_v into I_v and its child subtrees yields (4).

Conversely, sum (4) over the entire tree. Each nonroot flux term appears once positively and once negatively, leaving $Ax = 0$. Summing only over a rooted subtree gives (3). Thus no spurious scalar solutions survive, including on edges with zero intersection. This proves bijectivity. The additional claims follow because original coordinates and local forbidden pairs have not been altered.

Exact Gaussian elimination computes the two column-space bases and their intersection. Rational basis entries, intersection generators and their primitive integer scalings have polynomial encoding

length by determinant bounds. There are polynomially many cuts and local row records. An independent checker can compute the three ranks in (2), check that a proposed nonzero w_v lies in both column spaces, require zero exactly when the intersection has dimension zero, and compare every compiled row with (4). These are polynomial exact linear-algebra operations. □

Remark 4.2 (Rank zero and rejected cuts). The explicit zero-flux equations on rank-zero cuts are necessary. A zero coefficient w_v alone would leave an unrelated auxiliary scalar free. A cut with $\lambda_v > 1$ is rejected, not projected onto a selected direction. A forbidden pair crossing bags is likewise rejected, not silently dropped. Both rejection paths are exercised by the tests.

4.1 An exact and reusable source contract

The compiler accepts A, its declared number of columns, a partition into bags, a rooted tree, and the complete forbidden-pair list. The returned model retains original column names and the primitive cut generators. The source verifier uses rank-and-membership checks, not the producer's intersection-basis routine.

If a source column is later disabled, the same compiled system with that original coordinate set to zero remains exact. Rebuilding smaller cut spaces may improve performance, but is not required for correctness. If the source matching equations themselves change, such as after a Pachner move or crushing, a new source compilation or a separately proved transport certificate is required. An old small-width tree has no automatic authority over the new triangulation.

5 Source-bound gluing preserves extreme rays

An arbitrary signed scalar presentation can introduce artificial extreme rays. For example, present $\mathbb{R}^2_+$ using the redundant free scalar $\tau = a - b$. The chamber $\tau \geq 0$ has the ray $(a, b) = (1, 1)$, which is not extreme in $\mathbb{R}^2_+$. It is therefore not enough to observe that a local signed ray was chosen. The source-cut structure of Theorem 4.1 repairs this gap.

For each sign vector σ on the nonzero rank-one cuts, let

$$C_\sigma = \{x \in C(A) : \sigma_v \tau_v(x) \geq 0 \text{ for every such } v\}.$$

The scalars are now the unique source-bound linear functionals.

Lemma 5.1 (Zero flux splits the source vector). *If $x \in C(A)$ and $\tau_v(x) = 0$, the restrictions of x to S_v and to $\overline{S_v}$, extended by zero, both belong to $C(A)$. More generally, remove every zero-flux edge from the tree. Restricting x to the original columns of each remaining tree component gives feasible vectors whose sum is x. Each restricted vector belongs to every sign chamber containing x.*

Proof. Equation (3) gives $A_{S_v} x_{S_v} = 0$, and the complementary identity follows from $Ax = 0$. For several edges, use the compiled equations: retain a component's original coordinates and its internal fluxes and set all other coordinates and fluxes to zero. Its boundary fluxes were already zero, so every local equation remains true. The compiler theorem proves source feasibility and identifies the retained fluxes as the unique source fluxes. An internal flux keeps its sign, and an external one becomes zero. □

Theorem 5.2 (No new source rays under flux-sign subdivision). *Every extreme ray of every C_σ is an extreme ray of $C(A)$. Conversely, every extreme ray of $C(A)$ is an extreme ray of any C_σ containing it. Thus the union of the flux-sign chamber ray sets is exactly the source ray set. The same statements hold after any original coordinate-face restrictions.*

Proof. Let $x \neq 0$ span an extreme ray of C_σ. By Lemma 5.1, splitting the tree at the zero-flux edges decomposes x into feasible vectors in the same chamber with disjoint original supports. Extremality implies that only one tree component carries a nonzero vector. The source compiler's injectivity rules out a different component supported only by nonzero auxiliary fluxes.

Let F be the coordinate face of $C(A)$ obtained by setting every coordinate outside $\operatorname{supp} x$ to zero. The vector x lies in the relative interior of F: it is positive on all allowed support coordinates, so sufficiently small perturbations in $\operatorname{span} F$ preserve nonnegativity.

Every zero-flux edge separates the single occupied tree component entirely onto one side. For any $y \in F$, all its coordinates are on that side, and $Ay = 0$ forces the cut flux of y to be zero. Thus every sign inequality that is an equality at x is identically zero on F. All other sign inequalities are strict at x and remain valid under sufficiently small perturbations. If $\dim F \geq 2$, there is a vector $u \in \operatorname{span} F$ not parallel to x for which $x \pm \varepsilon u \in C_\sigma$ for small $\varepsilon > 0$. This contradicts extremality. Hence F has dimension one, so x is an extreme ray of $C(A)$.

The converse is immediate: a decomposition in the smaller cone would be a decomposition in the larger cone. Coordinate-zero restrictions commute with all the restrictions and zero extensions in the argument, proving the last assertion. □

Theorem 5.3 (Ray gluing and connected primitive witnesses)**.** *In the source-compiled model, suppose the local-ray recurrence chooses a single local extreme ray at each nonzero recipe node and uses the zero assignment on zero-port child subtrees. Every resulting nonzero global witness spans an extreme ray of the original matching cone, and satisfies the original compatibility constraints. If the source is a genuine standard normal-surface system, its primitive integer normalization represents a connected normal surface.*

Proof. Choose the global flux-sign chamber containing the reconstructed assignment. Consider a decomposition of this assignment into two vectors of that chamber, using the nonnegative variables obtained after fixing signs. At the origin node, the local vector is an extreme ray, so the two local summands are scalar multiples of it. Every nonzero child port forces the same scalar ratio at the child: its chosen local ray is extreme and its parent port is nonzero. This ratio propagates through all nonzero recipe edges. On a zero child subtree, all nonnegative chamber coordinates are zero, so both summands vanish there. Outside an early positive witness's origin subtree the same argument applies. Thus the full assignment is extreme in the compiled chamber. Projection is injective, and Theorem 5.2 makes the source vector extreme in $C(A)$. Original incompatibilities hold because they are local and every chosen ray satisfies them.

Scale the source vector to the primitive nonnegative integer vector p on its ray. Normal matching and compatibility give an embedded normal surface with coordinate vector p [4]. If it had two nonempty connected components, their coordinate vectors would be nonzero nonnegative integer vectors summing to p. Extremality makes each a nonnegative scalar multiple of p. An integer vector parallel to a primitive integer vector is an integer multiple: apply a Bezout combination of the coordinates of p, whose gcd is one. Each nonzero component would therefore contribute at least p, a contradiction. □

Remark 5.4 (Why the stronger conclusion is source-sensitive). The generic scalar solver is correct even for a model with redundant ports. Its positive output need not then be source-extreme. The code includes the $(a, b, a - b)$ counterexample and offers greedy original-support minimization as a safe fallback for a genuinely source-equivalent generic model. For a model accepted by the rank-one source compiler, the preceding theorem makes that extra search unnecessary. Independent rank tests on the positive support provide an inexpensive additional audit when the source matrix is available.

6 Bit complexity and the improvement over global branching

Let V be the number of tree vertices, $b_v = |I_v|$ the number of original coordinates at v, δ_v its degree, and $k_v = b_v + \delta_v$. For a fixed incident sign assignment σ, let $M_{v,\sigma}$ be its local equation matrix and put

$$d^{\mathrm{raw}}_{v,\sigma} = \dim \ker M_{v,\sigma}, \qquad d_{v,\sigma} = \dim \operatorname{span}\{z \geq 0 : M_{v,\sigma} z = 0\}.$$

These need not coincide: forced-zero coordinates can lower the intrinsic feasible dimension. Let N denote the total binary encoding length of the source, supplied decomposition, and objective; using the explicitly compiled model instead changes this length by at most a polynomial factor.

6.1 The delivered enumeration bound

Lemma 6.1 (Active-zero-set ray enumeration)**.** *For a rational matrix M with k columns and kernel dimension $d > 0$, every extreme ray of $\{z \geq 0 : Mz = 0\}$ is found by selecting $d-1$ coordinate functionals that are independent on $\ker M$, setting them to zero, and solving the resulting one-dimensional kernel. Testing all $\binom{k}{d-1}$ selections, checking signs and forbidden pairs, and removing duplicates is complete.*

Proof. At an extreme ray r, the face obtained by setting every coordinate outside $\operatorname{supp} r$ to zero has dimension one. On $\ker M$, those zero-coordinate functionals therefore have rank $d-1$. Select a basis of them. Their common kernel in $\ker M$ is exactly the ray's linear span, so one of the two orientations of the computed direction is r. Conversely, a nonzero nonnegative direction whose support-restricted kernel is one-dimensional spans an extreme ray. If $d = 1$, the empty selection is used. If $d = 0$, the cone is zero and there are no rays. □

Define the explicit local enumeration budget

$$\Xi = \sum_{v} \sum_{\sigma \in \{-1,1\}^{\delta_v}} \left(1 + \mathbf{1}_{\{d^{\mathrm{raw}}_{v,\sigma} > 0\}} \binom{k_v}{d^{\mathrm{raw}}_{v,\sigma} - 1}\right). \tag{5}$$

We use the convention $\binom{k}{-1} = 0$. The leading 1 charges the kernel computation even for an empty cone. The delivered code implements Lemma 6.1. It does not implement a polynomial-time LP engine or complete-support LP preprocessing.

Theorem 6.2 (Implemented exact-query bound)**.** *Compilation, local enumeration, scalar elimination, and construction of one positive witness have bit complexity*

$$\operatorname{poly}(N)\,\Xi.$$

In particular, with $h = \max_v(b_v + 2\delta_v)$, the cost is bounded by $\operatorname{poly}(N)2^h$. Thus a polynomial-size supplied source decomposition with $h = O(\log N)$ gives a polynomial query algorithm, and $h = O((\log N)^2)$ gives $N^{O(\log N)}$. After local preparation, further objectives and original coordinate deletions cost polynomial time in N and the number of retained local rays.

Proof. There are 2^{δ_v} local chambers at v. Each zero-set candidate uses polynomially many exact rational arithmetic operations on matrices of polynomial encoding length. Standard elimination and determinant bounds give polynomial bit length for every local kernel direction and primitive ray. Each candidate is either discarded or adds one ray, so all list sizes are charged to (5). The dynamic recurrence scans those lists a constant number of times per incident edge.

It remains to justify that rational message arithmetic does not escape the bound. The following lemma supplies a polynomial bound on message and witness bits. Finally, $\binom{k}{j} \le 2^k$, so summing over signs gives at most a polynomial factor times $2^{b_v+2\delta_v}$ at each node. Repeated queries only filter and rescore the already complete local lists. □

Lemma 6.3 (No hidden expansion in message arithmetic)**.** *Every finite message and every selected normalized witness has a rational encoding of polynomial length in N. The scalar algorithm need not enumerate normal discs or global compatibility histories to construct these numbers.*

Proof. One proof uses the normalized subtree problem from Theorem 3.2. After fixing its compatibility and scalar signs, it is a rational standard-form system of polynomial dimension with one parent coordinate set to 1 in absolute value. Whenever its finite optimum is attained, it is attained at a basic feasible solution. Clearing row denominators and applying Cramer's rule bounds numerator and denominator bits by a polynomial in the system's total encoding. The source compiler already has polynomial-size coefficients. The objective coefficients on original coordinates are part of the input; child slopes need not be included in this global subtree argument. Thus every exact optimum value has polynomial encoding length.

For the actual selected recipe, each local coordinate-to-parent ratio is a ratio of polynomial-size primitive-ray entries. A reconstructed coordinate is a product of such ratios along a simple tree path, times one origin-ray entry. There are at most V factors. Numerator and denominator bit lengths add under multiplication, so they remain polynomial. The objective is a sum of at most n products with the original rational coefficients. Reduced fractions in the implementation prevent irrelevant common factors from accumulating. □

This is a bit bound, not a unit-cost arithmetic argument. A gain such as 3^{1024} occupies only 1624 bits, while the associated number of admissible branch histories may be 2^{1024}. Neither quantity is used as an iteration bound in the scalar recurrence.

6.2 A sharper local intrinsic-dimension theorem

The following refinement uses the complete-support and dimension-drop principles already available in report 87 [7]. It is useful as a target for a future LP-backed implementation.

Lemma 6.4 (Compatibility cover by intrinsic dimension)**.** *A nonnegative rational cone of intrinsic dimension d with forbidden coordinate pairs admits a complete binary compatibility search with at most $2^{d+1} - 1$ nodes, using polynomial-time rational LP at each node. On each compatible leaf, feasibility or linear optimization under a specified normalization is an ordinary LP problem.*

Proof. At a node, compute which coordinates can be positive in the current cone. This can be done by one feasibility query per coordinate, or by the complete primal–dual support certificate of report 87. If no forbidden pair has both coordinates live, every point of the cone is compatible. Otherwise choose a forbidden pair i, j with both coordinates live. Adding their positive witnesses shows that they can be simultaneously positive in the convex cone.

Branch on $x_i = 0$ or $x_j = 0$. Every compatible point belongs to one child. Each coordinate functional is nonzero on the current feasible span; hence each child has intrinsic dimension at least one smaller. The depth is at most d, and the node bound follows. All nodes use the original rational equations plus coordinate zeros. Polynomial-time rational LP [5] gives polynomial bit cost per node. Normalized optimizations are performed only after a compatible leaf is obtained, so no objective-dependent branching assumption is needed. □

Theorem 6.5 (Sum of local intrinsic-dimension costs)**.** *The scalar-tree positive-objective problem can be decided in bit time*

$$\mathrm{poly}(N)\sum_{v}\sum_{\sigma\in\{-1,1\}^{\delta_v}} 2^{d_{v,\sigma}}.$$

This is a sum over local searches, not the product of their sizes. With $D=\max_{v,\sigma} d_{v,\sigma}$ *and* $\Delta=\max_v \delta_v$*, the bound is* $\mathrm{poly}(N)2^{D+\Delta}$*. In particular,* $D+\Delta=O((\log N)^2)$ *is a quasi-polynomial criterion even when the global matching-cone dimension is linear in* N*.*

Proof. At a node and sign chamber, unavailable child directions and disabled source coordinates give coordinate faces and cannot increase intrinsic dimension. Use Lemma 6.4 to cover the compatible local cone. On each leaf, test for a positive zero-parent-port direction and optimize the effective objective with parent absolute value one when feasible. If an optimum is unbounded, its recession direction supplies the already decisive zero-port positive witness. Theorem 3.2 then computes the exact parent message from these local outcomes.

Each subtree is processed once, and its compatibility history is replaced by its message before the parent is solved. There is no enumeration of tuples of leaf choices from different subtrees. Lemma 6.3 bounds the intermediate coefficients, giving the displayed bit cost. The final estimate sums at most $V2^{\Delta}$ chamber costs bounded by 2^{D}. □

To retain the connected-witness conclusion with an LP backend, extract a vertex of an optimal normalized local face rather than averaging tied optima. A positive-port vertex corresponds to a cone ray; a positive zero-port search can use a sum-of-coordinates normalization. Vertex extraction inside a rational optimal face uses polynomially many exact LP or linear-algebra operations. The delivered ray backend already chooses a single ray, including when several rays have the same score.

6.3 A strict separation from a global search budget

Consider m stages with nonnegative variables $t_0,\dots,t_m,u_i,v_i$ and

$$t_{i-1}=u_i+v_i,\qquad t_i=2u_i+3v_i,\qquad u_iv_i=0\quad(1\le i\le m). \tag{6}$$

At each stage, the only admissible nonzero choices transmit a gain of 2 or 3. Consecutive stages share one nonnegative scalar. Eliminating the intermediate t_i into scalar interfaces, and retaining t_0 and t_m as endpoint coordinates, gives the disjoint-coordinate model used in the code. The eliminated values are uniquely recovered and are automatically nonnegative. Their objective coefficients can be absorbed into the adjacent stage equations. The model has two endpoint rays and two admissible rays per stage.

Proposition 6.6 (Exponential global ray family, linear local inventory)**.** *The unrestricted matching cone in* (6) *has dimension* $m+1$*. Its admissible extreme rays are in bijection with the* 2^m *binary gain histories. Nevertheless, every linear objective on its original variables can be tested for positivity by the scalar recurrence with* $2m+2$ *retained local rays and polynomial bit cost.*

Proof. There are $3m+1$ variables and $2m$ independent equations. Independence follows successively from the variables $t_m,t_{m-1},\dots$ and the stage equations, or by parameterizing the linear solution space using t_0 and $v_1,\dots,v_m$. The cone has a strictly positive point, obtained by taking $u_i=v_i>0$ at every stage, so its intrinsic dimension is the same $m+1$.

For a nonzero compatible vector, $t_0>0$: if $t_0=0$, nonnegativity forces $u_1=v_1=0$, hence $t_1=0$, and induction makes the entire vector zero. Normalize $t_0=1$. At each stage exactly one of u_i,v_i is positive, yielding its binary history. Fixing that history makes the matching solution unique

up to scale, so the corresponding coordinate face is a ray. Different histories have different original supports even when their final gains happen to agree. The local ray counts and the scalar algorithm now prove the final assertion. □

For the objective $t_m - 3^m t_0$, the answer is nonpositive; changing the coefficient to $3^m - 1$ gives a positive witness. The family provides a strict comparison with a flat complete enumeration of all compatibility histories. It is also analytically soluble by the elementary maximum-gain formula. It is not offered as a difficult knot family, a lower bound on every algorithm, or a novel closed-form optimization problem.

7 A sharp warning for two-coordinate interfaces

The success of one scalar does not justify summarizing a two-dimensional interface by a fixed number of slopes. The obstruction already occurs in a constant-width binary counter.

Theorem 7.1 (Exponential literal cone messages in interface dimension two)**.** *For every $m \geq 1$, there is a homogeneous rational system with $O(m)$ variables and constraints, coefficients bounded by 2 in absolute value, and local forbidden pairs, whose interface projection onto (s, w_m) is exactly*

$$P_m = \bigcup_{k=0}^{2^m-1} \{(s, ks) : s \geq 0\}.$$

Any representation of P_m as a finite union of convex cones requires at least 2^m nonzero cones. The 2^m directions are distinguishable by homogeneous linear contexts with a positive-objective query.

Proof. Take $s, u_i, v_i, w_i \geq 0$, fix $w_0 = 0$, and impose

$$u_i + v_i = s, \qquad u_i v_i = 0, \qquad w_i = 2w_{i-1} + v_i \quad (1 \leq i \leq m).$$

When $s > 0$, each v_i is either 0 or s. Thus w_m/s is an arbitrary m-bit integer, and every integer between 0 and $2^m - 1$ occurs. When $s = 0$, all coordinates vanish. This proves the projection formula. Consecutive counter stages communicate only the pair (s, w_i), so the phenomenon persists with two-dimensional separators and bounded local equations.

If a convex cone contained nonzero vectors on two different displayed rays, it would contain their positive combinations. Their slopes range over an interval, including slopes not in the finite set of integers. Therefore a convex cone contained in P_m meets at most one nonzero displayed ray. At least 2^m cones are required.

For a specified integer k, attach the homogeneous equation $w_m = ks$ and ask whether the objective s is positive. This context accepts exactly the kth direction. Its coefficient has only $O(m)$ bits. □

The lower bound is explicitly about a *literal union of convex cones*. The counter itself is a short symbolic description, so the theorem cannot be used to rule out arithmetic circuits, automata, semilinear descriptions, or specialized exact optimization in rank two. It pinpoints a representation barrier, not an unrestricted complexity lower bound.

8 Connection to complete unknot recognition

8.1 The standard-coordinate interface

For a validated triangulation with t tetrahedra, the standard coordinates have four triangle and three quadrilateral entries per tetrahedron. The three forbidden pairs among each quadrilateral triple enforce admissibility. Matching equations, a source-derived rational Euler functional, and an original triangle coordinate set to zero fit our homogeneous problem directly.

The classical positive-Euler/crushing recognition framework searches a one-vertex knot-exterior triangulation for a connected positive-Euler normal surface other than the vertex link; an essential-boundary disc proves unknottedness, while the other permitted outcomes are handled by the geometric reduction [3]. Our source-bound ray theorem supplies connected witnesses for that interface. It does not replace the essential-boundary test or the proof that the triangulation is the exterior of the input knot.

In a one-vertex triangulation, the vertex-link vector has every triangle coordinate equal to one. A connected normal surface different from the vertex link has some triangle coordinate zero: otherwise a vertex-link copy can be removed near the vertex as a disjoint normal component. Thus the search can use the disjunction of at most $4t$ zero-triangle queries. Primitivity and connectedness are important here; merely producing a positive Euler sum would allow an irrelevant vertex-link component to dominate the objective.

For a genuine knot exterior, an embedded connected positive-Euler surface is a sphere or a disc: an embedded projective plane does not occur in the 3-sphere. A sphere, a boundary-parallel disc, and an essential-boundary disc must still be distinguished by the geometric layer. The generic algebraic status POSITIVE is deliberately not named UNKNOT.

8.2 An explicit conditional running-time criterion

Theorem 8.1 (Width-controlled recognition criterion)**.** *Consider a correct positive-Euler/crushing recognition implementation operating on a diagram of binary input size N_0. Assume the following contracts.*

(i) Source construction, one-vertex conversion, geometric checking, and reduction have total polynomial bit cost outside their positive-Euler queries. The number and encoding sizes of query sources are polynomial in N_0.

(ii) A decomposition producer, including all failed proposals, has total cost $N_0^{O(\log N_0)}$ and returns source-verified, conflict-local rank-one tree models for every query, or equivalent coordinate restrictions with authenticated extension back to the source.

(iii) Across these query models, the implemented budget satisfies $\sum \Xi \le N_0^{O(\log N_0)}$.

Then replacing the positive-Euler search by the delivered scalar ray method preserves complete recognition and gives total running time $N_0^{O(\log N_0)}$. A sufficient local condition for (iii) is polynomially many bags with $b_v + 2\delta_v = O((\log N_0)^2)$. For a polynomial-time LP implementation, the alternative sufficient condition is $d_{v,\sigma} + \delta_v = O((\log N_0)^2)$.

Proof. Source compilation preserves every query. Scalar elimination is complete in both directions, and selected primitive rays give connected source surfaces. The recognition implementation therefore receives exactly the required kind of answer and continues through its already correct geometric steps. The query costs sum by Theorems 6.2 and 6.5; the producer cost and all nonquery costs are included explicitly. The displayed sufficient conditions follow by substituting the local bounds. □

A single bag containing every source column always gives an exact model with no interface condition. This supplies a complete exponential fallback at the algebraic query level. It does not give the width assumption of the theorem. Neither the one-vertex condition, normal matching, nor a small triangulation dual-treewidth by itself has been proved here to imply that assumption.

The classical strictly decreasing tetrahedron count gives a particularly clear special case: at most t reduction stages and at most $4t$ triangle anchors per stage give $O(t^2)$ anchored queries. Prepared local rays can be reused across anchors on a fixed source. Reuse across a changed source is not covered.

8.3 Relation to Lackenby's hierarchy program

The scalar theorem is a search primitive, not a substitute for the entire hierarchy. To use it for later hierarchy pieces, the source constraints must still express the correct surfaces and inherited boundary conditions. A marked-boundary objective or charge may introduce several interface quantities, nonhomogeneous conditions, or cross-bag restrictions. Such changes must be proved compatible with the message contract rather than inferred from a small raw-cut encoding.

In particular, a proof of short hierarchy depth does not itself prove small scalar width, and small scalar width at one stage does not itself control hierarchy resets. The most useful next connection would be a source-aware normal form for the hierarchy pieces in which most of the matching system splits by rank-one cuts and only a polylogarithmic residual core remains. That is a concrete additional theorem to seek, not a conclusion established by the present experiments.

9 Implementation, verification, and experiments

9.1 Delivered components

The Python package has four implementation modules. `exact.py` provides rational Gaussian elimination, nullspaces, primitive integer normalization, active-zero-set ray enumeration, and a separate positive-support oracle. `model.py` validates tree incidence and private coordinate ownership, compiles source cuts, eliminates auxiliary ports for small independent audits, and independently replays source compilations. `solver.py` prepares local sign-chamber rays, evaluates messages, reconstructs witnesses, and checks positive or negative packets. `fixtures.py` supplies reproducible algebraic models and source-derived matching and Euler arithmetic for two repository triangulations.

All mathematical arithmetic uses Python integers and `Fraction`. There is no floating-point feasibility decision, no external LP solver, and no Regina runtime dependency. Floating-point values occur only in timing measurements. The core API is:

```
model, generators = compile_source(A, n, bags, parents, pairs)
assert verify_source_compilation(
    A, n, bags, parents, pairs, model, generators)
prepared = Prepared(model)
answer = prepared.solve(objective, zeros=disabled_columns)
assert verify(model, objective, answer, zeros=disabled_columns)
```

For this source API, original column names are their zero-based decimal strings. The complete executable example is in the package README. The caller must separately authenticate the source matrix and Euler functional to a valid triangulation and the triangulation to the knot diagram.

The ray enumerator is intentionally transparent rather than optimized for large dense bags. There is no implicit cutoff: the caller must not submit a huge binomial budget accidentally.

Production integration should add explicit budgets and cancellation, returning an inconclusive status on exhaustion. Exhaustion must never be translated into NONPOSITIVE or a knot verdict.

9.2 Verification design

The unit suite has **27 passing test methods**. Included are zero-dimensional cones, forced-zero coordinates, rank deficiency, large rational coefficients, 100 seeded random comparisons of the two local ray enumerators, positive zero-port shortcuts, unavailable signs, negative fluxes, objective changes, coordinate deletion reuse, primitive support minimization, malformed trees, rank-two cut rejection, cross-bag conflict rejection, incorrect source generators, omitted negative rays, altered slopes, corrupted positive coordinates, incorrect objectives, and invalid face reciprocity.

The two local enumeration methods are algorithmically different. The producer chooses $d-1$ vanishing coordinate rows in a kernel parameterization. The checker chooses a positive support in the original coordinates and requires the restricted matching kernel to have dimension one. They share the elementary rational elimination implementation, so this is not a formally verified or fully independently implemented arithmetic stack. That common dependency is an explicit residual risk.

9.3 Independent global-oracle audit

Using seed 12102026, the experiment constructs 96 small scalar-tree models with one to four vertices, one or two private coordinates per vertex, signed local equations, and local forbidden pairs. For each model it eliminates all ports to obtain a global source matrix, enumerates that source's admissible rays by positive supports, and tests eight exact rational objectives. Four of the eight also allow additional randomly chosen original-coordinate deletions.

The resulting **768 queries** comprise **455 positive** and **313 nonpositive** outcomes. Every outcome agrees with the independent global oracle. The same source is then compiled afresh from its column cuts, and all 768 queries are repeated on that source-bound model. Each source compilation passes the separate rank-and-membership checker.

Both the original and recompiled packets are independently replayed, for **1,536 packet replays**. Every one of the 455 positive source-compiled vectors also passes a direct extremality test:

$$\dim \ker A_{\operatorname{supp} x} = 1.$$

The audit therefore tests the additional source-bound ray theorem, not merely agreement of Boolean answers. There are 384 coordinate-face queries in the 768-query set. Raw cases, objectives, deleted columns and statuses are saved in `evidence/results.json`; the generation schedule is reproducible.

These are finite algebraic checks. They do not prove the theorems, which have separate proofs above, and they are not a sample of 96 knot exteriors.

9.4 Actual normal-coordinate fixtures

The first fixture is the repository's one-tetrahedron solid torus: faces 0 and 1 are identified by the vertex permutation $(1,2,3,0)$ and its inverse, while faces 2 and 3 remain boundary faces. The second attaches a tetrahedral ball along face 2 using permutation $(0,1,3,2)$ [9]. The matching adapter constructs, for every identified face and every face corner, the equality of its normal-arc counts. It validates reciprocal face-pairing data, but not all manifold-link conditions.

The Euler adapter uses $V-E+F$: a local normal disc contributes one face, a normal arc contributes one half when its source face is glued and one when it lies on the boundary, and a

normal edge point contributes the reciprocal of the number of local tetrahedron edges in its source edge class. The two known meridian vectors have Euler characteristic one under this functional.

The one-tetrahedron matrix has rank 3 and four admissible rays. The 14-coordinate capped matrix has rank 6 and twelve admissible rays. In each case the full local ray set agrees with independent global support enumeration. There are 21 coordinate-positivity probes and twelve zero-triangle Euler queries over the two fixtures. In the first fixture, the meridian query returns

$$(1, 1, 0, 0, 0, 0, 1),$$

exactly the repository's expected standard vector. All supplied positive vectors are checked in the original matching equations.

The capped example should not be confused with a ready-made one-vertex recognition benchmark: its extra tetrahedron can change the vertex inventory. The anchored Euler predicates are valid algebraic queries, but their positive answers are not individually advertised as unknot verdicts.

9.5 A genuine source split, and a rejected geometric split

Splitting the capped fixture by its two tetrahedra gives cut-space dimension *three*, not one. The compiler correctly rejects that proposal. A single shared source face has three normal-arc types; geometric adjacency alone does not supply a scalar interface.

A different, nongeometric column partition does have rank one:

$$I_1 = \{0, 1, 10\}, \qquad I_0 = \{2, 3, 4, 5, 6, 7, 8, 9, 11, 12, 13\}.$$

All quadrilateral triples remain in I_0, so no incompatibility crosses the partition. With the repository-derived matching-row order, the verified intersection generator is

$$w_1 = (1, 0, -1, 1, 1, 0)^T.$$

This split groups three triangle coordinates; it is a matrix decomposition, not a cut of the triangulation into the two displayed bags.

The one-bag ray preparation examines 3,432 active-zero-set candidates and emits 12 admissible rays. The split preparation examines 1,856 candidates and emits 13 local rays in total. The latter number is larger: the improvement is in local candidate search, not in a claim that every local inventory is smaller than the final global ray set.

	One source bag	Verified scalar split
Candidate zero sets	3,432	1,856
Retained local rays	12	13
Anchored Euler queries	8	8
Median complete kernel time (s)	0.660599	0.340946

Table 1: Seven paired runs on the actual capped source fixture. Timed scope includes matching and Euler construction, source compilation and rank replay, ray preparation, all eight queries, and positive arithmetic verification.

The median paired one-bag/split ratio is **1.938-fold**. An identical-code one-bag control gives median control/one-bag ratio 1.016. Run order was independently shuffled in each of the seven rounds. Every arm returned the same eight Boolean outcomes; witnesses need not be identical. This is a source-local performance gain in the delivered implementation, not a comparison with ProveIt's optimized native routines, not a whole-knot benchmark, and not evidence for a universal asymptotic width bound.

9.6 Algebraic scaling and small-input overhead

The gain-chain benchmark compares the complete supplied-model query with literal enumeration of all binary gain histories. Each timed kernel call constructs the model, enumerates local rays, and executes the scalar search. Independent certificate replay is outside both timed arms. All benchmark objectives are $t_m - 3^m t_0$, so both methods must certify nonpositivity rather than benefiting from a first positive witness. Seven pairs and an identical-code kernel control are used at each size.

m	Histories	Local rays	Kernel (ms)	Literal (ms)	Paired gain
1	2	4	0.576	0.001	0.002×
4	16	10	2.258	0.007	0.003×
8	256	18	3.981	0.125	0.033×
12	4,096	26	6.657	2.685	0.420×
16	65,536	34	8.495	55.049	6.480×
18	262,144	38	10.503	244.919	23.764×

Table 2: Algebraic gain-chain timings. "Paired gain" is the median of the seven literal/kernel ratios, not the ratio of the two reported medians. The literal baseline is complete history enumeration, not the best algorithm for this analytically soluble family.

The smallest cases strongly favor literal traversal. At $m = 18$, the kernel handles 38 retained local rays rather than 262,144 histories and achieves a median paired gain of 23.76-fold. This is the expected crossover for a high-overhead exact rational method replacing an exponential enumeration. The unfavorable small cases are retained in the evidence instead of selecting only the favorable endpoint.

m	Local rays	Candidate sets	Max. slope bits	Seconds
64	130	1,028	102	0.089
256	514	4,100	406	0.130
1024	2,050	16,388	1,624	0.631

Table 3: Larger supplied algebraic chains. These are single unpaired scaling runs, not stable comparative timing estimates. There are 2^m admissible histories, none explicitly enumerated by the kernel.

The largest run uses 1,024 stages, 2,050 local rays, and 16,388 candidate zero sets. Its largest message numerator occupies 1,624 bits. These counts provide a direct check that large multiplicities remain binary arithmetic rather than loop bounds. No claim is made that these chains are realized by the tested normal triangulations.

The two-port counter is separately checked for all stages $1 \le m \le 12$: its binary histories give every integer slope in the specified interval, without omissions or repetitions. The finite check supports the construction; the all-m lower bound is proved in Theorem 7.1.

9.7 Limits of this validation

No complete local clone of ProveIt was available, the incoming ZIP bodies were not inspected, Regina was not installed, and the native maintained suite was not run. None of the upstream README's large test counts are claimed as tests of this package. No GitHub branch, issue, or pull request was created or changed.

The delivered negative verifier is complete but expensive for large bags. The source compiler is polynomial for a supplied partition but does not discover one. The prototype has no production cancellation interface. The small source benchmark does not establish a performance distribution over knot diagrams. Finally, no Lean formalization is included; the assurance consists of the written proofs, exact source contracts, executable checks, and disclosed finite experiments.

10 Integration plan and research priorities

10.1 An additive integration path

The package should first be kept as a self-contained research module under the repository's preferred report or research directory. Its public statuses are algebraic. Promoting it into `fastunknot` should require an adapter that obtains the actual source matching matrix, Euler functional, coordinate map, quadrilateral exclusions, and source identity from maintained producers. Every compiled split should be replayed before use.

A practical dispatcher can retain the one-bag or existing search as fallback, try inexpensive verified scalar splits, compare their predicted enumeration budgets, and use the smaller route only when it improves the forecast. The forecast should charge 2^{δ_v} sign chambers, raw versus effective nullity, and all source-preparation overhead. The capped-fixture result shows why merely counting retained rays is not a sufficient dispatch heuristic.

Prepared local lists can be reused across the triangle anchors and across linear objectives on the same source. Original-coordinate deletion is safe; source mutation requires recompilation. The positive verifier can remain much smaller than the search producer. Negative coverage must include every local chamber and ray or an equally complete LP certificate tree.

Before a default-path promotion, the repository should require fresh native comparisons on its maintained normal-surface corpus, source-authenticated essential-disc checks, forced budget exhaustion tests, malformed-source tests, and paired benchmarks against the incumbent dispatcher. A prototype's positive Euler status must not bypass those geometric checks.

10.2 Further mathematical questions

1. Conflict-preserving source decompositions. Develop a polynomial-time producer for useful rank-one column-cut trees while requiring each quadrilateral triple to remain local. Ordinary represented-matroid separation algorithms do not, by themselves, enforce this extra locality rule. The relevant deliverable is a decomposition with a proved cost measure, or a certified obstruction explaining why a large residual core is unavoidable.

2. Rank spectra on genuine knot sources. Measure cut-space dimensions after maintained support propagation, complete primal–dual support reduction, and controlled compatibility decisions. The rank-three tetrahedron split in the capped fixture is an immediate warning against inferring algebraic width from geometric adjacency. Which source operations create long rank-one regions, and which preserve large rank cores?

3. A small-core theorem compatible with dense normal vectors. A useful global theorem need not bound the number of nonzero normal coordinates. It could instead assert that the source matching system decomposes into many scalar pieces plus a residual core with polylogarithmic intrinsic dimension. Can parallelity or hierarchy normal forms provide this property without enumerating their exponentially many sheets?

4. Source-sensitive higher-rank ray preservation. The no-new-rays argument uses the fact that vanishing scalar flux means vanishing full cut vector. In higher rank, a sign boundary usually kills only one linear functional of the cut vector. Identify arrangements whose boundary strata still split the source support into independent coordinate pieces. This would extend connected-witness guarantees beyond rank one.

5. Succinct rank-two messages. Theorem 7.1 rules out a uniformly small literal cone list, but not a short symbolic language. Investigate arithmetic circuits, monotone ratio intervals with discrete exceptional sets, or finite-state descriptions carrying binary counters. The objective is an exact composition theorem with bit and state bounds, not a convexification that fills forbidden slopes.

6. LP-backed local search. Implement the intrinsic-dimension theorem using a polynomial-time exact LP policy or a certificate-producing proposal backend with exact replay. Compare it with the delivered active-zero-set enumeration on the same supplied trees. Separate the mathematical polynomial LP guarantee from the practical behavior of any chosen simplex pivot rule.

7. Amortization across anchors and reductions. Coordinate-zero queries already share prepared rays. Can source-bound dual certificates and rank-one generators be transported cheaply across the specific crushing or Pachner transformations used by the recognizer? A transport rule must preserve source equations and forbidden-pair locality, not just matrix rank or Euler values.

8. Width-aware decomposition optimization. For a given family of admissible cuts, minimize the actual sum of chamber budgets rather than tree depth alone. An empty internal bag can still have a large degree and many sign chambers. Determine when balancing a tree preserves rank-one cuts and yields a demonstrable net saving.

9. More than one objective. A fixed Euler objective collapses to two slopes, but retaining several objectives may require a Pareto frontier. Characterize cases in which essential boundary data, homology, or a lexicographic secondary weight can be carried without recreating exponentially many continuation states.

10. Nonhomogeneous boundary conditions. The zero-extension shortcut fails in the presence of fixed nonzero boundary charges. Find a precise replacement using normalized rooted messages or inhomogeneous value functions, with an explicit bound on their breakpoints. This is necessary before transferring the scalar method indiscriminately to marked hierarchy pieces.

11. Native adversarial validation. Build a reproducible corpus in which the incumbent normal-sector search is actually expensive, then test verified source splits under identical source preparation and certificate obligations. Include sources that reject all nontrivial scalar splits, small cases with overhead, high-bit coordinate examples, and deliberately invalid source transports.

12. The remaining quasi-polynomial theorem. Prove a bounded-cost complete source/decomposition family for every required recognition query, or a higher-rank continuation theorem that covers the large-core cases. The hierarchy length, pattern data, source bit growth, decomposition search, and negative coverage must all be charged together. These are the explicit missing hypotheses between the local theorems proved here and the unrestricted target.

11 Conclusion

The principal improvement is not a faster way to enumerate the same global compatibility histories. It is an exact reason not to enumerate their Cartesian product: a homogeneous one-scalar continuation has only two finite optimization slopes once a decisive zero-port direction is excluded. The source compiler makes that interface testable from the original matching equations, and the no-new-rays theorem turns the resulting local choices into primitive connected normal witnesses without a second global support search.

This gives a rigorous search-width criterion, a tested exact implementation, and a small genuine source-level performance gain. It also exposes a sharp boundary of the approach: two-coordinate interfaces can already carry exponentially many distinct conic directions, and even one geometric face can induce a rank-three matching interface. The next advance should therefore be sought in source-aware decomposition or succinct higher-rank continuation, with the complete recognition costs kept explicit.

A Reproduction and file map

From the unpacked package root, run:

```
python -m unittest discover -s tests -v
python experiments/reproduce.py --out /tmp/scalar-separator-evidence
python experiments/build_tables.py
cd article
latexmk -pdf -interaction=nonstopmode -halt-on-error article.tex
```

The table generator reads the delivered `evidence/results.json`; it does not silently replace it with a new timing run. The optional output path on the reproduction script permits comparisons while preserving the frozen evidence.

Path	Contents
`article/`	This PDF, complete editable TeX, generated tables and macros.
`scalar_separators/`	Exact arithmetic, source models, source replay, message solver, packet verifier, and fixtures.
`tests/test_kernel.py`	The 27-method standard-library regression suite.
`experiments/reproduce.py`	Seeded global-oracle audit, actual matching fixtures, source split benchmark, algebraic scaling, and rank-two counter.
`experiments/build_tables.py`	Tables generated from the frozen JSON.
`evidence/results.json`	Environment, generated cases, exact counts, source data and individual paired timing records.
`evidence/benchmark.csv`	Compact gain-chain timing table.
`SOURCE_AUDIT.md`	Reference checkpoint, inspected paths, bibliography, and uninspected incoming archives.
`CLAIMS.md`	Proven, conditional, implemented and unvalidated claims, kept separate.
`INTEGRATION.md`	Source authentication, fallback semantics, native integration gates and limitations.

Path	Contents
`validation/`	Fresh unit-test and reproduction logs and PDF checks.

The package does not include a dependency snapshot of the full repository and does not need one to run its own tests. Reproducing a future native integration will require the actual maintained source producers and corpus, not merely these two fixture definitions.

References

[1] M. Lackenby, *Incompressible surfaces, hierarchies and unknot recognition*, arXiv:2607.23350v1 (2026), especially Sections 9, 10 and 14. `https://arxiv.org/abs/2607.23350`.

[2] M. Lackenby, *A quasi-polynomial time algorithm for unknot recognition*, lecture slides (2021), linked from the author's publication page. `https://people.maths.ox.ac.uk/lackenby/`.

[3] B. A. Burton and M. Ozlen, *A fast branching algorithm for unknot recognition with experimental polynomial-time behaviour*, arXiv:1211.1079v3 (2014), Sections 2–3. `https://arxiv.org/abs/1211.1079`.

[4] W. Haken, *Theorie der Normalflächen*, Acta Mathematica 105 (1961), 245–375.

[5] L. G. Khachiyan, *A polynomial algorithm in linear programming*, Soviet Mathematics Doklady 20 (1979), 191–194.

[6] S. Borgwardt, W. Grewe and J. Lee, *On the Combinatorial Diameters of Parallel and Series Connections*, arXiv:2203.09587v3 (2023), introduction and Section 1.1. `https://arxiv.org/abs/2203.09587`.

[7] ProveIt, report 87, *Certified reductions for unknot recognition*, especially `article/support_theory.tex` and `article/search_budget.tex` (2026). Inspected source path: `https://github.com/VladimirReshetnikov/ProveIt/tree/4840e384b5a2961aabbb9cc78023b9ac5990acb0/Topology/UnknotRecognition/reports/87`.

[8] ProveIt, *Full three-dimensional normal cuts in affine cell families*, `Topology/UnknotRecognition/synthesis/cut_cells.tex` (2026). Repository: `https://github.com/VladimirReshetnikov/ProveIt`.

[9] ProveIt, `Topology/UnknotRecognition/fast/tests/test_normal_sector_integration.py`, source fixtures and geometric scope tests (2026). Repository as above.